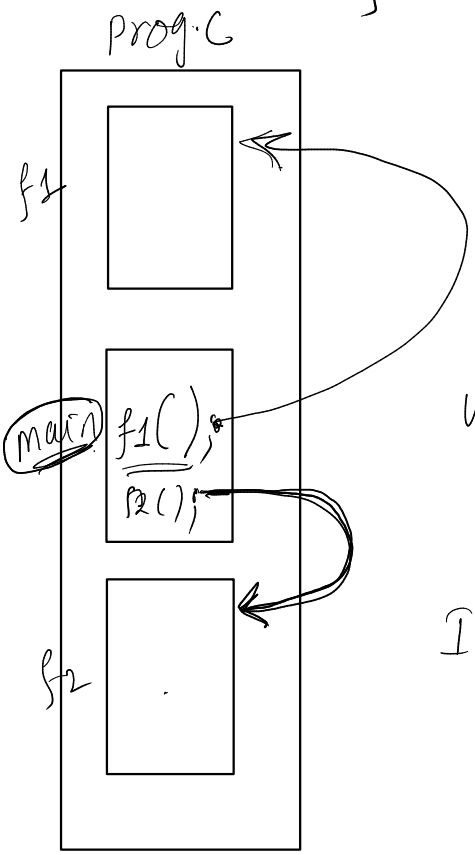
Function (Decl, defn, call)

→ Self contained block, dedicated for a specific task.

```
ret_type fun_name(Arg Declaration)
{
  ==
}
```

```
int checkdigit(int V, int d)
{
  ==
  ==
}
```

definition/Body



✓ What happens in a function call?

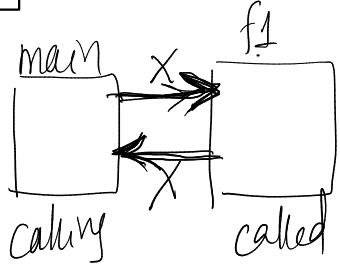
⇒ —— How?

what happens after all stmts in a function executed?

or, →

Is function Declaration must?

↳ as Identifier, it is supposed to be declared



However, in some cases declaration NOT needed:

```
void f1(void)
{
}

main()
{
  f1(); //call
}
```

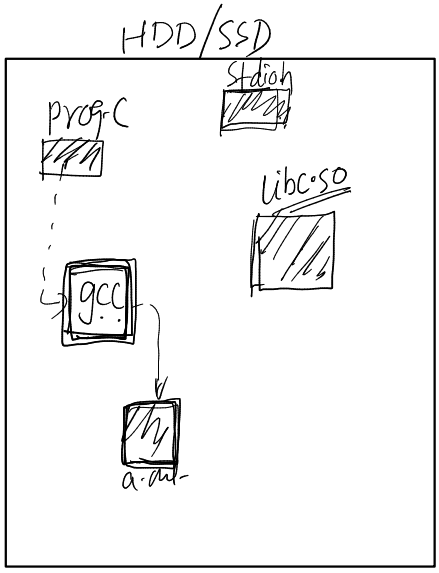
on this, which function translated first?

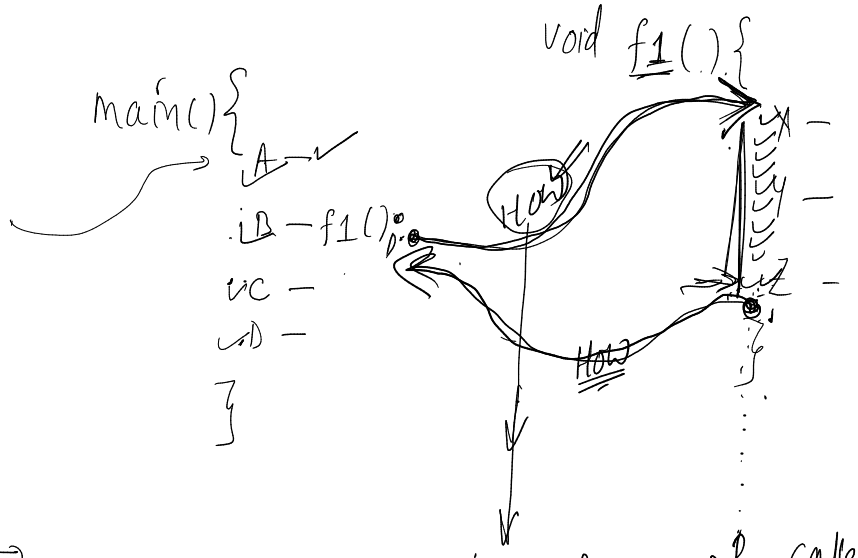
Two types of functions

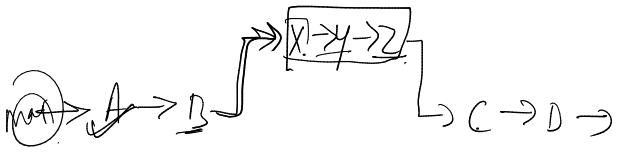
- Library function - later
- User-defined function

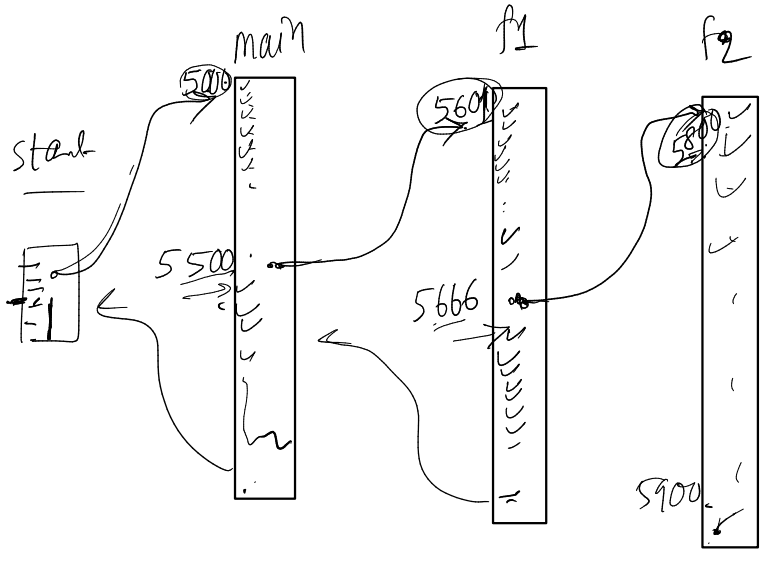
.dll

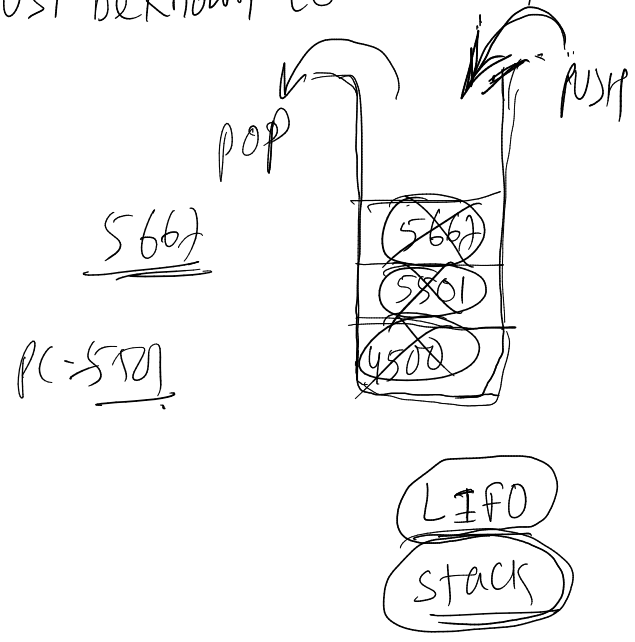
- → Code Reusability
- → Readable
- → Easy to debug
- → Complex problem can be solved easily by dividing to sub-problems





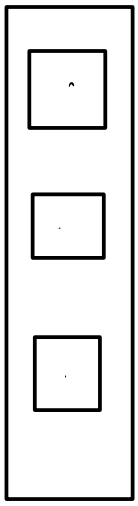




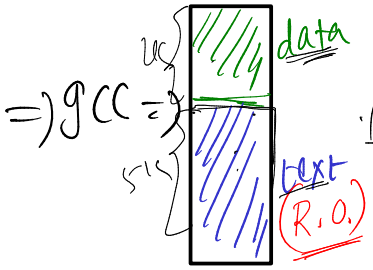


Memory partitions in an App

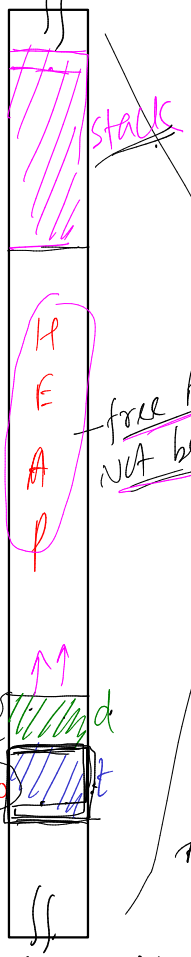
Prog.C



a.out



=) gcc



=) A src. prog contains one/more functions

=) An executable contains — at least two segments

CPU

i) text seg/code seg: contains Bin. eq. of the program

ii) Data seg: contains — Global and static variables

After loaded into memory —

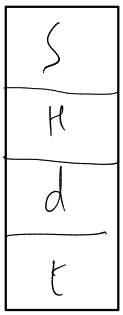
stack seg. is assigned to the Job

text seg Size ∝ Amt of source code

data seg Size ∝ No. of Global and static variables

```
main() {
    int x=10;
    static int y=20;
    f1(x+y);
    =
}
```

```
z->ntr
f1( int z)
{
    30
    =
    =
    =
}
```



stack Seg : contains auto variables, formal Arguments, and return Addr

: Size decided by OS, not affected by No-of-variables.